

# GUIDE.md --- Complete Project Guide

## PS 19: Implement Reinforcement Learning for Game Optimization

**Institution:** Madhav Institute of Technology and Science (MITS), Gwalior

**Domain:** Reinforcement Learning / Game AI

**Games:** Snake--Apple Game, Flappy Bird, Pac-Man

**Primary implementation basis:** Milestones 1, 2, and the official Milestone 3 supplied by the project team

**External implementation resources:** Gymnasium, Atari/ALE documentation, Flappy Bird Gymnasium, and selected RL research papers

---

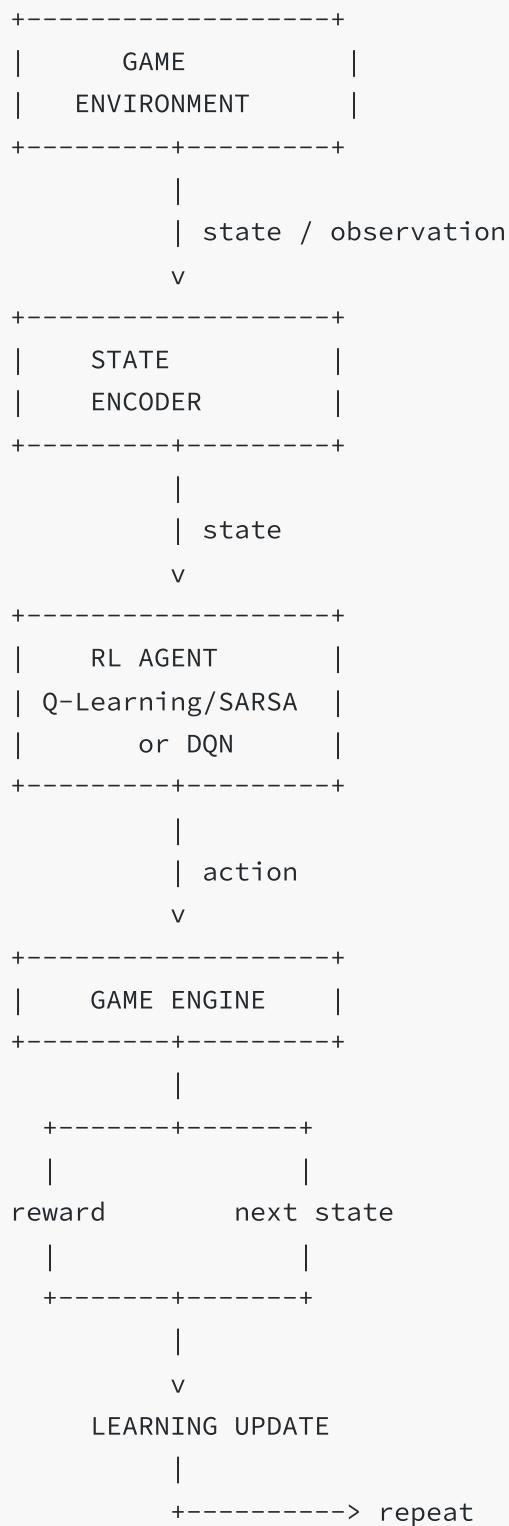
### 0. READ THIS FIRST

This guide is the single working guide for the complete project.

The project is **not** about writing a hard-coded bot that follows a fixed optimal path.

The project is about building agents that:

## REINFORCEMENT LEARNING



The central question is:

How do state representation, reward design, exploration, and RL algorithm affect learning and performance across three different games?

The project therefore needs **reproducible experiments**, not just one impressive gameplay video.

---

# 1. PROJECT IN ONE PAGE

## 1.1 What are we building?

Three RL game agents:

---

| Game | Main agent in State | Main purpose | official Milestone 3 |
|------|---------------------|--------------|----------------------|
|------|---------------------|--------------|----------------------|

---

|              |             |             |                           |
|--------------|-------------|-------------|---------------------------|
| Snake--Apple | Refined DQN | (4, 64, 64) | Deep RL with visual state |
|--------------|-------------|-------------|---------------------------|

|             |                          |                   |                                       |
|-------------|--------------------------|-------------------|---------------------------------------|
| Flappy Bird | Q-Learning + 75 discrete | Tabular on-policy | SARSA states vs off-policy comparison |
|-------------|--------------------------|-------------------|---------------------------------------|

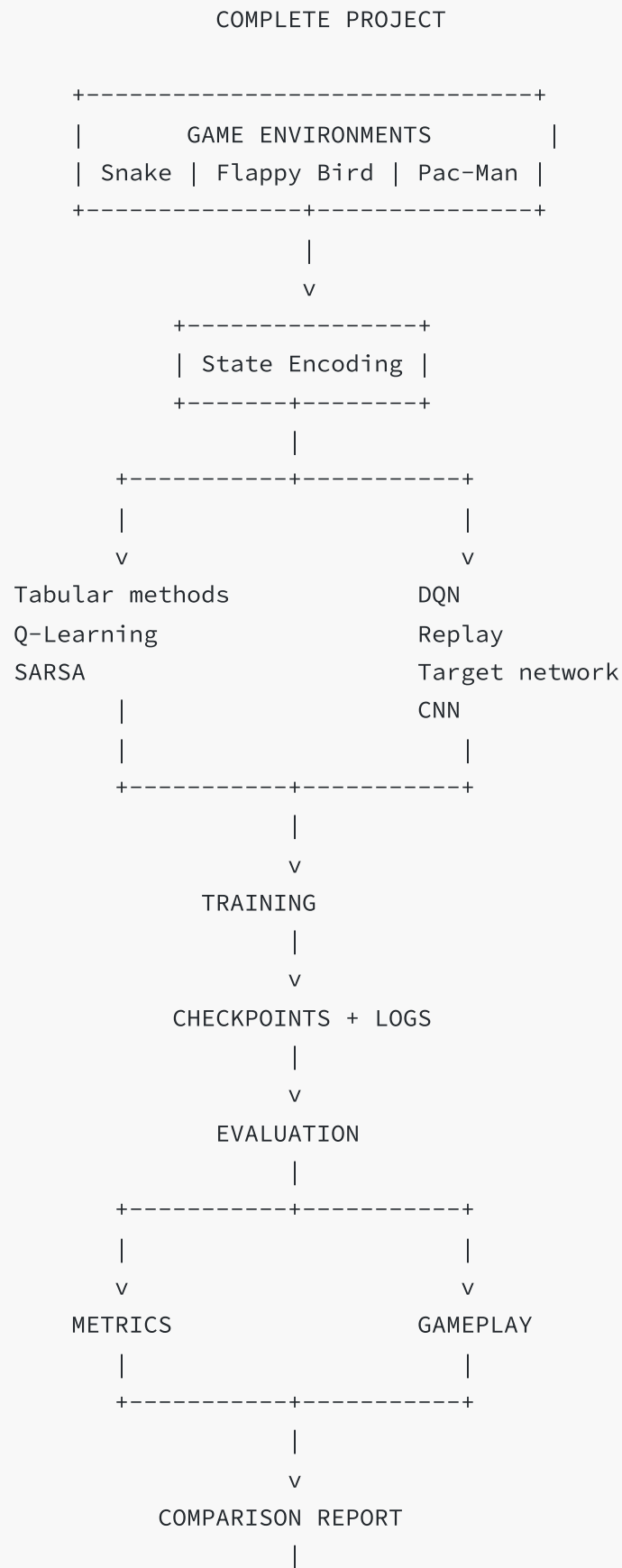
|         |     |             |                                     |
|---------|-----|-------------|-------------------------------------|
| Pac-Man | DQN | (4, 84, 84) | Deep RL on visual Atari-style state |
|---------|-----|-------------|-------------------------------------|

---

The project-level methodology also treats Q-Learning and SARSA as the core tabular methods and DQN as the function-approximation extension when explicit Q-tables become impractical.

---

## 1.2 What should happen when the project is finished?



## 2. THE MOST IMPORTANT RULES

### Rule 1 --- Environment comes first

Do not start by writing DQN.

First make sure the game can reliably:

```
reset
|
v
observe
|
v
accept action
|
v
advance game
|
v
return reward + next state + terminal
```

Milestone 2 explicitly makes environment construction and verification the immediate goal before training conclusions.

---

### Rule 2 --- Keep environment and agent separate

The environment should not contain DQN logic.

Bad:

```
SnakeEnvironment
-> contains replay buffer
-> contains neural network
-> selects actions
```

Good:

```
SnakeEnvironment
|
+--> reset()
+--> observe()
+--> step(action)
+--> reward
+--> terminal
    ^
    |
    SnakeDQN
```

This allows the same environment to be tested independently and keeps experiments reproducible.

---

### Rule 3 --- Do not silently change the paper-defined environment

The Pac-Man research basis is specifically **Atari 2600 Ms. Pac-Man**.

If another Pac-Man implementation is used, record it as:

```
PROJECT MODIFICATION
```

Do not claim that it is the exact paper environment.

---

### Rule 4 --- Training and evaluation are different

Training:

```
exploration ON
epsilon > minimum
learning ON
parameters changing
```

Evaluation:

```
exploration OFF or controlled
learning OFF
parameters frozen
fixed evaluation episodes
```

Never report a training episode as if it were a clean evaluation result.

---

## **Rule 5 --- Never conclude from one lucky episode**

The project should report:

- mean
- standard deviation
- distributions where useful
- learning curves
- survival/episode length
- success rate
- native game score
- representative gameplay
- failure cases

A single best score is not sufficient evidence of learning.

---

## **3. HOW TO START --- SHORT PRACTICAL BRIEF**

If you are completely new to the project, follow this order:

PART 0

Project setup

|

v

PART 1

Common RL infrastructure

|

v

PART 2

Build + verify Snake

|

v

PART 3

Train Snake DQN

|

v

PART 4

Build + verify Flappy Bird

|

v

PART 5

Train Flappy Q-Learning + SARSA

|

v

PART 6

Build + verify Pac-Man

|

v

PART 7

Train Pac-Man DQN

|

v

PART 8

Unified evaluation

|

v

PART 9

Ablation + failure analysis

|

v

PART 10

Final plots + report + demo



## Why this order?

### Snake first

- Simple game mechanics.
- Small grid.
- Easy to understand reset/action/reward/terminal behavior.
- Good first DQN environment.
- Useful for learning preprocessing, frame stacking, replay, checkpointing, and debugging.

### Flappy Bird second

- Very small action space.
- Official project state has only 75 discrete states.
- Ideal for understanding Q-Learning and SARSA before dealing with a large visual DQN.

### Pac-Man last

- More complicated visual state.
  - Atari environment.
  - Larger strategic state.
  - DQN and preprocessing introduce more moving parts.
  - Better attempted after the team understands the complete RL pipeline.
- 

## 4. MILESTONE ROADMAP

The three milestones should be treated as a dependency chain.

#### MILESTONE 1

Problem + MDP + algorithm strategy

|

v

#### MILESTONE 2

Environment construction + verification

|

v

#### MILESTONE 3

Agent implementation + training

|

v

#### FINAL EXPERIMENTS

Evaluation + comparison + ablation + failure analysis

---

## Milestone 1 --- Design

Milestone 1 establishes:

- project objective
- MDP formulation
- common architecture
- Q-Learning
- SARSA
- DQN
- epsilon-greedy exploration
- state engineering
- reward design
- training strategy
- evaluation plan
- risks
- reproducibility

It identifies:

- Snake as a growing-state game with delayed consequences.
- Flappy Bird as a timing-sensitive game with a small action space.

- Pac-Man as a larger visual/adversarial environment where tabular state storage is difficult.
- 

## Milestone 2 --- Environment

Milestone 2 defines the common environment contract:

```
RESET
|
v
OBSERVE
|
v
ACT
|
v
STEP
|
+-----> REWARD
|
+-----> NEXT STATE
|
+-----> TERMINAL
```

It requires environment smoke tests, sample observations, reward traces, episode traces, and an integration run.

Milestone 2 does not claim project performance results.

---

## Official Milestone 3 --- Agents + Training

The official Milestone 3 is the implementation basis for the training stage:

```
Snake          -> Refined DQN
Flappy Bird    -> Q-Learning + SARSA
Pac-Man        -> DQN
```

It specifies:

- state dimensions
- network structures
- replay configuration
- target-network updates

- epsilon schedules
  - rewards
  - logging
  - checkpointing
  - initial evaluation
  - debugging/validation
  - deliverables
  - transition to final comparison
- 

## 5. COMMON RL CONCEPTS

### 5.1 State

The state is what the agent receives about the current game situation.

Examples:

Snake:

image history

Flappy:

distance + vertical position + velocity

Pac-Man:

processed image history

---

### 5.2 Action

The action is the control selected by the agent.

Examples:

Snake:  
UP / DOWN / LEFT / RIGHT

Flappy:  
FLAP / DO NOT FLAP

Pac-Man:  
controller directions/actions

---

## 5.3 Reward

Reward tells the agent whether the recent transition was useful.

Conceptually:

```
good event    -> positive reward
bad event     -> negative reward
survival      -> small positive reward when defined
```

Reward design must be fixed before final comparison.

---

## 5.4 Episode

One game attempt:

```
RESET
|
v
PLAY
|
v
DEATH / TERMINATION
|
v
EPISODE END
```

---

## 5.5 Return

The agent aims to maximize future reward:

```
G_t = r_t + gamma*r_(t+1) + gamma^2*r_(t+2) + ...
```

`gamma` controls the importance of future rewards.

---

## 6. Q-LEARNING

Q-Learning is an **off-policy** temporal-difference method.

Update:

```
Q(s,a) <- Q(s,a)
  + alpha[
    r + gamma * max Q(s',a')
    - Q(s,a)
  ]
```

Meaning:

```
current Q
|
+--> observe reward
|
+--> look at best estimated next action
|
+--> calculate target
|
+--> update Q
```

Important:

Q-Learning is off-policy.

---

## 7. SARSA

SARSA is an **on-policy** temporal-difference method.

Update:

```
Q(s,a) <- Q(s,a)
  + alpha[
    r + gamma * Q(s',a')
    - Q(s,a)
  ]
```

The next action `a'` is the action actually selected by the behavior policy.

Sequence:

```
S
|
A
|
R
|
S'
|
A'
|
v
UPDATE
```

Important:

SARSA is on-policy.

## 8. DQN

DQN replaces the giant Q-table with a neural network:

```
state
|
v
+-----+
| CNN / Neural Net |
+-----+
|
v
Q(s,a1) Q(s,a2) ... Q(s,an)
```

DQN in this project uses:

- neural Q-function approximation
- experience replay
- target network
- epsilon-greedy exploration

DQN is **off-policy**.

---

## 9. EPSILON-GREEDY

During training, the agent sometimes explores.

```
random number < epsilon
    |
    YES
    |
    v
    random action

random number >= epsilon
    |
    NO
    |
    v
    best Q action
```

At the beginning:

```
epsilon = high
    |
    v
    more exploration
```

Later:

```
epsilon = low
    |
    v
    more exploitation
```

Evaluation should use a frozen policy with exploration disabled or controlled according to the protocol.

---



## 10. DQN COMPONENTS

### 10.1 Online network

Produces current Q estimates:

```
state -> ONLINE NETWORK -> Q values
```

### 10.2 Target network

Provides a more stable target.

```
state'
 |
 v
TARGET NETWORK
 |
 v
target Q
```

The target network is periodically updated from the online network.

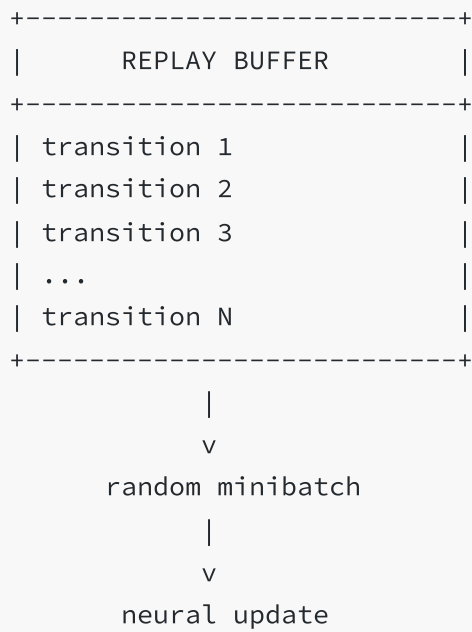
---

### 10.3 Experience replay

Each interaction produces:

```
(s, a, r, s', done)
```

Stored in:



Replay breaks strong temporal correlation and lets the agent reuse experience.

---

## 11. COMMON TRAINING LOOP

Every agent should conceptually follow:



---

## 12. PRACTICAL PROJECT FOLDER STRUCTURE

Use a single repository with common infrastructure and separate game implementations.

rl-game-optimization/

- README.md
- GUIDE.md
- LICENSE
- requirements.txt
- pyproject.toml
- .gitignore
- configs/
  - global.yaml
  - snake/
    - env.yaml
    - dqn.yaml
  - flappy/
    - env.yaml
    - q\_learning.yaml
    - sarsa.yaml
  - pacman/
    - env.yaml
    - dqn.yaml
- src/
  - common/
    - \_\_init\_\_.py
    - seed.py
    - logger.py
    - metrics.py
    - checkpoint.py
    - evaluation.py
    - utils.py
  - environments/
    - \_\_init\_\_.py
    - base\_env.py
    - snake\_env.py
    - flappy\_env.py
    - pacman\_env.py
  - preprocessing/
    - \_\_init\_\_.py

```
├── snake_preprocess.py
├── flappy_state.py
├── pacman_preprocess.py
├── frame_stack.py
├── agents/
│   ├── __init__.py
│   ├── tabular/
│   │   ├── q_learning.py
│   │   └── sarsa.py
│   └── dqn/
│       ├── replay_buffer.py
│       ├── target_network.py
│       ├── networks.py
│       ├── dqn_agent.py
│       └── snake_refined_replay.py
├── training/
│   ├── train_q_learning.py
│   ├── train_sarsa.py
│   ├── train_snake_dqn.py
│   └── train_pacman_dqn.py
├── evaluation/
│   ├── evaluate_q_learning.py
│   ├── evaluate_sarsa.py
│   ├── evaluate_snake.py
│   ├── evaluate_pacman.py
│   └── compare_agents.py
├── visualization/
│   ├── learning_curves.py
│   ├── loss_curves.py
│   ├── score_distribution.py
│   ├── q_statistics.py
│   └── comparison_plots.py
├── games/
│   ├── snake/
│   │   ├── README.md
│   │   └── assets/
```

```
├── flappy/
│   ├── README.md
│   └── assets/
├── pacman/
│   ├── README.md
│   └── assets/
├── notebooks/
│   ├── 00_setup_smoke_test.ipynb
│   ├── snake/
│   │   ├── 01_snake_environment.ipynb
│   │   ├── 02_snake_preprocessing.ipynb
│   │   ├── 03_snake_dqn_training.ipynb
│   │   └── 04_snake_evaluation.ipynb
│   ├── flappy/
│   │   ├── 01_flappy_environment.ipynb
│   │   ├── 02_flappy_state_discretization.ipynb
│   │   ├── 03_flappy_q_learning.ipynb
│   │   ├── 04_flappy_sarsa.ipynb
│   │   └── 05_flappy_evaluation.ipynb
│   └── pacman/
│       ├── 01_pacman_environment.ipynb
│       ├── 02_pacman_preprocessing.ipynb
│       ├── 03_pacman_dqn_training.ipynb
│       └── 04_pacman_evaluation.ipynb
├── tests/
│   ├── test_common.py
│   ├── test_snake_env.py
│   ├── test_flappy_env.py
│   ├── test_pacman_env.py
│   ├── test_q_learning.py
│   ├── test_sarsa.py
│   └── test_dqn_components.py
├── checkpoints/
│   ├── snake/
│   ├── flappy/
│   └── pacman/
```

```
├── models/
│   ├── snake_dqn_best.pt
│   ├── flappy_q_table.pkl
│   ├── flappy_sarsa_table.pkl
│   └── pacman_dqn_best.pt
├── logs/
│   ├── snake/
│   ├── flappy/
│   └── pacman/
├── evaluations/
│   ├── snake/
│   ├── flappy/
│   └── pacman/
├── plots/
│   ├── snake/
│   ├── flappy/
│   ├── pacman/
│   └── comparisons/
├── videos/
│   ├── snake/
│   ├── flappy/
│   └── pacman/
├── reports/
│   ├── environment_validation.md
│   ├── training_report.md
│   ├── evaluation_report.md
│   ├── ablation_report.md
│   ├── failure_analysis.md
│   └── reproducibility_report.md
└── metadata/
    ├── environment_versions.json
    ├── experiment_registry.json
    └── seeds.json
```

---

## 13. WHY THIS FOLDER STRUCTURE IS DYNAMIC

The structure separates:



```
GAME LOGIC
  from
STATE PROCESSING
  from
AGENTS
  from
TRAINING
  from
EVALUATION
  from
RESULTS
```

Therefore a future change such as:

```
Snake DQN
  |
  +--> Double DQN
  |
  +--> Dueling DQN
  |
  +--> another replay strategy
```

does not require rewriting the game environment.

---

## 14. FILE RESPONSIBILITIES

### **src/environments/**

Contains only environment behavior.

Example:

```
state = env.reset()
next_state, reward, done, info = env.step(action)
```

Do not put neural-network training here.

---

### **src/preprocessing/**

Converts raw game observations into the exact representation required by the agent.

Example:

```
raw frame
  |
  v
grayscale
  |
  v
resize
  |
  v
normalize / mask
  |
  v
frame stack
  |
  v
agent state
```

---

### **src/agents/**

Contains learning algorithms.

```
tabular/
  q_learning.py
  sarsa.py

dqn/
  replay_buffer.py
  networks.py
  dqn_agent.py
```

---

### **src/training/**

Contains episode loops and experiment execution.

---

### **src/evaluation/**

Contains frozen-agent evaluation.

---

### **configs/**

Do not scatter hyperparameters throughout Python files.

Store them centrally.

---

## 15. ENVIRONMENT CONTRACT

All environments should expose the same conceptual contract:

```
reset()
|
+--> initial observation

step(action)
|
+--> next observation
+--> reward
+--> terminal
+--> info
```

A conceptual base interface:

```
+-----+
|   RL Environment   |
+-----+
| reset()            |
| observe()          |
| step(action)       |
| action_space       |
| observation_space  |
+-----+
```

Gymnasium currently uses a `reset()` / `step()` interface where `step()` returns observation, reward, termination/truncation information, and `info`.

When adapting a modern Gymnasium environment to the project's simpler conceptual `done` flag, handle `terminated` and `truncated` explicitly rather than silently ignoring one.

---

## 16. PART 1 --- COMMON PROJECT SETUP

### Step 1 --- Create the repository

Create:

```
rl-game-optimization/
```

Initialize Git.

---

## Step 2 --- Create a Python environment

Use a virtual environment or Conda environment.

Recommended baseline:

```
Python  
NumPy  
PyTorch  
Gymnasium  
Matplotlib  
Pandas
```

Additional packages depend on the selected game environments.

---

## Step 3 --- Install the core framework

Use the official Gymnasium documentation as the primary installation/API reference:

<https://gymnasium.farama.org/>

Use the official repository when checking source-level details:

<https://github.com/Farama-Foundation/Gymnasium>

---

## Step 4 --- Run a smoke test

Before building a game:

```
import framework  
create environment  
reset environment  
inspect observation  
select random legal action  
step environment  
print reward  
print termination
```

The purpose is only:

INSTALLATION

|

v

ENVIRONMENT CREATION

|

v

RESET

|

v

STEP

|

v

SUCCESS

---

## 17. PART 2 --- BUILD SNAKE FIRST

### 17.1 Why Snake first?

Snake gives the team a relatively understandable environment while still introducing:

- visual state
- frame history
- collision logic
- reward design
- replay
- DQN
- checkpointing
- evaluation

It is a practical first complete deep-RL pipeline.

---

## 18. SNAKE ENVIRONMENT SPECIFICATION

The supplied Snake research describes:

- Python implementation
- 240 x 240 pixel map

- 12 x 12 grid
- initial snake length 3
- initial direction right
- random snake/apple deployment
- one apple at a time
- snake grows after eating
- new apple is spawned after eating
- controls UP/DOWN/LEFT/RIGHT
- termination on wall/self collision

Source basis: Milestone 2, Snake environment section.

---

## 18.1 Snake state flow

```
240 x 240 game
    |
    v
12 x 12 logical grid
    |
    v
rendered frame
    |
    v
RGB image
    |
    v
resize to 64 x 64
    |
    v
HSV/background processing
    |
    v
preprocessed frame
    |
    v
4-frame history
    |
    v
64 x 64 x 12
```

The official Milestone 3 input notation is:

```
(4, 64, 64)
```

Conceptually this represents four processed grayscale/masked frames. The environment-level description in Milestone 2 explains the intermediate four-frame stacked representation as `64 x 64 x 12` after RGB preprocessing; keep the implementation convention explicit so the tensor layout is never ambiguous.

---

## 19. SNAKE ACTIONS

Use:

```
0 = UP  
1 = DOWN  
2 = LEFT  
3 = RIGHT
```

Keep the mapping in one configuration/source file.

Never rely on unexplained numeric actions scattered across notebooks.

---

## 20. SNAKE REWARD

The Snake paper describes:

```
apple eaten      -> +1  
collision        -> -1
```

with rewards clipped to:

```
[-1, +1]
```

It also describes:

- distance reward
- training gap after eating an apple
- timeout penalty
- two experience sets

These are part of the research basis.

The official Milestone 3 refined DQN configuration uses:

```
apple      = +1.0
survival   = +0.1
collision   = -1.0
```

Therefore the implementation should explicitly distinguish:

```
paper/environment reward description
      |
      v
official Milestone 3 training configuration
```

Do not accidentally mix the two.

---

## 21. SNAKE REFINED DQN

Official Milestone 3 configuration:



Input:

(4, 64, 64)

Convolution:

16 filters

32 filters

64 filters

Fully connected:

256 units

Head:

Dueling DQN

Replay:

high-TD-error buffer = 35,000

low-TD-error buffer = 15,000

total = 50,000

TD threshold:

0.5

Sampling:

70% high-error

30% low-error

Batch:

32

Learning rate:

0.0001

Gamma:

0.99

Target update:

every 500 steps

Training gap:

K = 4 frames

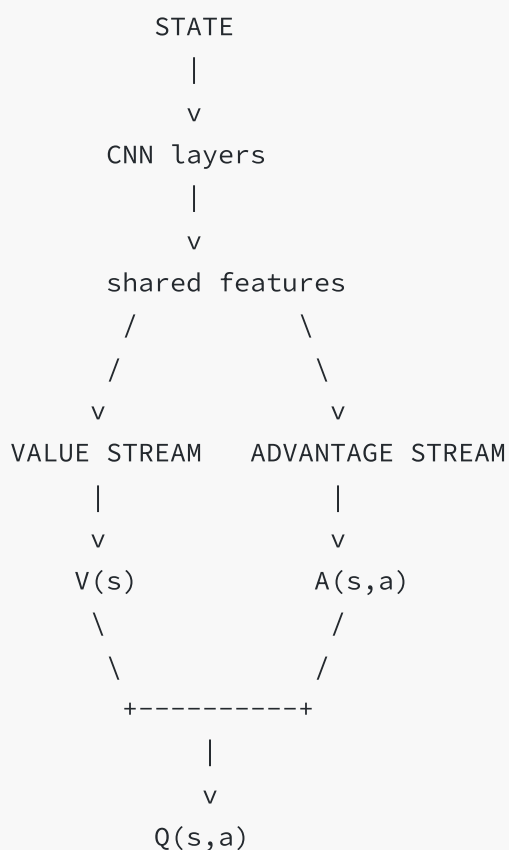
Epsilon:

1.0 -> 0.02

Epsilon decay:  
200,000 frames

## 22. SNAKE DUELING NETWORK

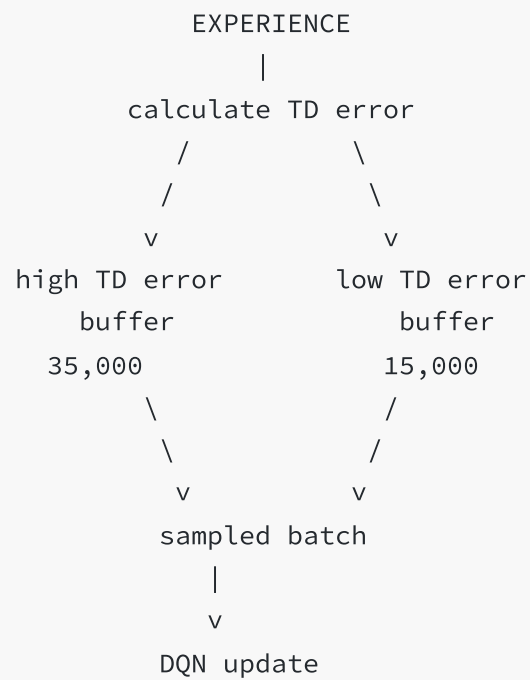
Conceptually:



The project uses a dueling head in the official Milestone 3 Snake configuration.

## 23. SNAKE DUAL REPLAY

The refined Snake implementation separates experiences into two groups:



Sampling target:

70% high-error  
30% low-error

The purpose is to make informative experiences more likely to influence learning.

## 24. SNAKE TRAINING PROCEDURE

Build in this exact order:

1. Build game  
|
2. Verify reset  
|
3. Verify actions  
|
4. Verify collision  
|
5. Verify apple reward  
|
6. Verify observation shape  
|
7. Implement preprocessing  
|
8. Implement frame stack  
|
9. Implement CNN  
|
10. Implement replay  
|
11. Implement target network  
|
12. Implement epsilon-greedy  
|
13. Train short run  
|
14. Inspect logs  
|
15. Resume from checkpoint  
|
16. Run longer training  
|
17. Freeze checkpoint  
|
18. Evaluate

---

## 25. SNAKE VALIDATION GATES

Before training:

```
[ ] reset produces valid initial game
[ ] snake length = 3
[ ] initial direction = right
[ ] snake/apple deployment is valid
[ ] apple is present
[ ] movement changes position correctly
[ ] actions map correctly
[ ] wall collision terminates
[ ] self collision terminates
[ ] apple produces correct reward
[ ] state dimensions are correct
[ ] frame stack is correct
[ ] terminal episode resets correctly
```

Save evidence in:

```
reports/environment_validation.md
logs/snake/environment_smoke_test.csv
```

---

## 26. PART 3 --- BUILD FLAPPY BIRD

After Snake is functioning, move to Flappy Bird.

---

## 27. FLAPPY BIRD ENVIRONMENT

The supplied FlapAI Bird paper uses an OpenAI Gym Flappy Bird environment.

The environment exposes information about:

- nearest pipes
- bird position
- bird velocity
- screen images

The paper uses different representations for different agents.

---

### 27.1 Tabular state

For Q-Learning/SARSA, use:

```
horizontal distance  
vertical distance  
bird y-velocity
```

The official Milestone 3 discretizes this into:

```
x bins = 5  
y bins = 5  
velocity bins = 3  
  
total states = 5 * 5 * 3  
              = 75
```

Therefore:

```
state = (x_bin, y_bin, velocity_bin)
```

---

## 28. FLAPPY BIRD ACTIONS

Official mapping:

```
0 = flap  
1 = do not flap
```

Keep this mapping consistent everywhere.

---

## 29. FLAPPY BIRD REWARD

Official Milestone 2 / Milestone 3 project reward:

```
survive      -> +0.5  
pass pipe    -> +5  
collision    -> -1000
```

The `+0.5` survival reward is intended to encourage continued survival rather than random behavior between pipes.

---

## 30. FLAPPY BIRD Q-LEARNING

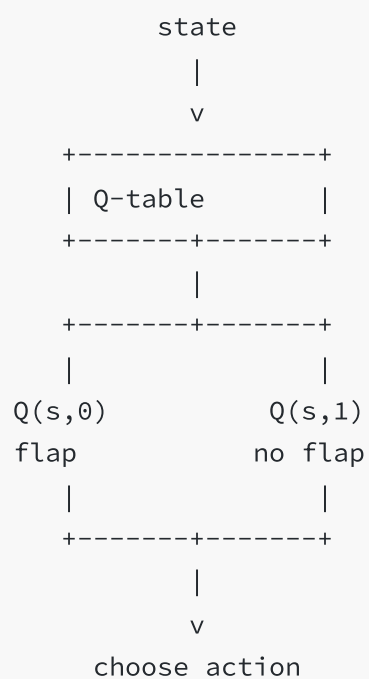
Create:

```
Q[state][action]
```

with:

```
75 states  
2 actions
```

Conceptually:



## 31. FLAPPY BIRD SARSA

SARSA uses the actual next action selected by epsilon-greedy.

```
state
|
v
choose A
|
v
step environment
|
v
receive R and S'
|
v
choose A' using behavior policy
|
v
SARSA update
```

This makes Flappy Bird the clearest place in the project to demonstrate:

OFF-POLICY  
Q-Learning

vs.

ON-POLICY  
SARSA

---

## 32. FLAPPY EPSILON SCHEDULE

Official Milestone 3:

```
epsilon:
    1.0 -> 0.05

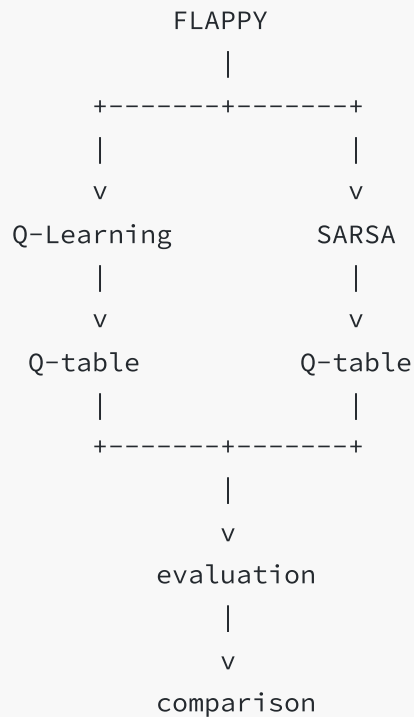
decay:
    over 5000 episodes
```

---

## 33. FLAPPY TRAINING ORDER

Train:





Do not change the state representation between the two algorithms during the main comparison.

---

## 34. FLAPPY VALIDATION

Before training:

```
[ ] environment starts
[ ] reset returns valid state
[ ] x-distance available
[ ] y-distance available
[ ] velocity available
[ ] discretization works
[ ] exactly 75 possible state combinations
[ ] action 0 = flap
[ ] action 1 = no flap
[ ] pipe pass reward works
[ ] survival reward works
[ ] collision reward works
[ ] terminal state works
[ ] Q-table indexing works
```

---

## 35. PART 4 --- BUILD PAC-MAN LAST

Pac-Man should be attempted after the other pipelines are stable.

---

## 36. PAC-MAN ENVIRONMENT BOUNDARY

The supplied Pac-Man paper is specifically:

```
Atari 2600 Ms. Pac-Man
```

The paper treats the visual environment as partially observable.

Therefore:

```
raw game
  |
  v
screen observation
  |
  v
preprocessing
  |
  v
frame history
  |
  v
DQN
```

If another Pac-Man implementation is used, record the deviation.

---

## 37. PAC-MAN OBSERVATION

The project uses:

```
4-frame state
84 x 84 spatial resolution
```

Official Milestone 3 notation:

```
(4, 84, 84)
```

The Milestone 2 description starts from an:

84 x 84 colour image

and produces a heavily downsampled black-and-white representation for the DQN.

Therefore the preprocessing pipeline must be explicit.

Suggested conceptual flow:

```
Atari observation
  |
  v
crop / preprocess as configured
  |
  v
84 x 84
  |
  v
grayscale / binary-style representation
  |
  v
4-frame stack
  |
  v
(4,84,84)
```

Do not silently replace this with a different representation.

---

## 38. PAC-MAN ACTIONS

Use the legal controller actions exposed by the selected Atari/Ms. Pac-Man environment.

Keep the action mapping documented in:

configs/pacman/env.yaml

The project must not assume that the action index mapping is identical across every Atari environment wrapper.

---

## 39. PAC-MAN DQN

Official Milestone 3:

Input:

(4, 84, 84)

CNN:

32 -> 64 -> 64 channels

Kernel:

3 x 3

Stride:

1

Fully connected:

512

Head:

Dueling

Actions:

4

---

## 40. PAC-MAN REPLAY CONFIGURATION

Official Milestone 3:

Replay capacity:

100,000

Batch:

32

Learning rate:

0.00025

Gamma:

0.99

Target update:

every 1000 steps

---

## 41. PAC-MAN EPSILON

Official Milestone 3:

```
epsilon:  
    1.0 -> 0.01  
  
decay:  
    500,000 frames
```

The Milestone 2 Pac-Man paper description also discusses an epsilon schedule beginning at 1.0, decreasing toward approximately 0.1 during training, with a lower value during evaluation. The official Milestone 3 project configuration should control the actual implementation.

---

## 42. PAC-MAN REWARD

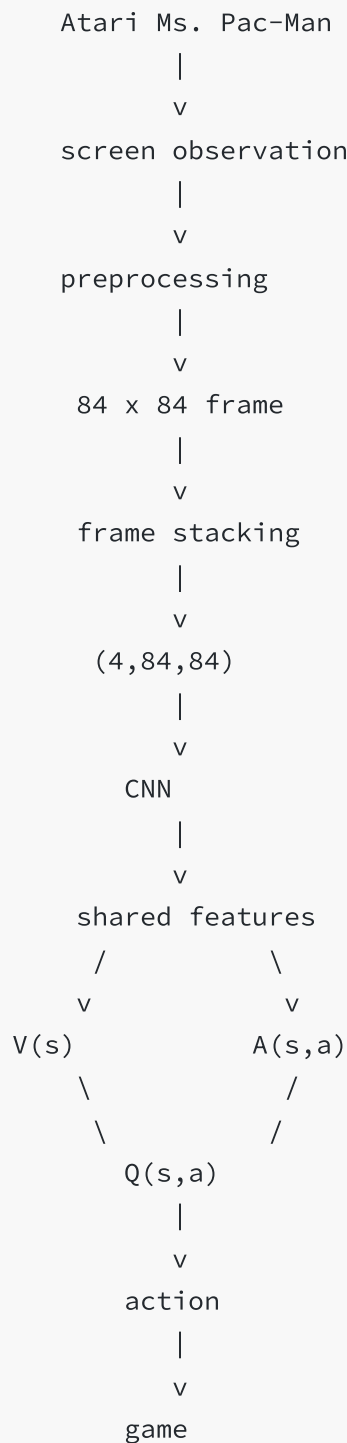
Official Milestone 3 configuration:

```
pellet      -> +0.1  
power       -> +0.5  
step        -> -0.01  
ghost       -> -50
```

Keep these constants in the Pac-Man configuration rather than hard-coding them throughout the environment.

---

## 43. PAC-MAN DQN FLOW



## 44. PART 5 --- COMMON TRAINING INFRASTRUCTURE

Once each environment is independently validated, standardize training infrastructure.

## 45. CONFIGURATION MANAGEMENT

Every experiment should record:

```
game
agent
state representation
reward configuration
learning rate
gamma
alpha where applicable
epsilon start
epsilon end
epsilon decay
batch size
replay size
target update
training gap
episode count
seed
environment version
code version
```

Example:

```
game: snake
agent: refined_dqn

seed: 42

state:
  shape: [4, 64, 64]

optimizer:
  learning_rate: 0.0001

gamma: 0.99

epsilon:
  start: 1.0
  end: 0.02
  decay_frames: 200000

replay:
  high_error_capacity: 35000
  low_error_capacity: 15000
  td_threshold: 0.5
  high_fraction: 0.70
  low_fraction: 0.30

batch_size: 32

target_update_steps: 500

training_gap: 4
```

---

## 46. LOGGING

Official Milestone 3 requires logging of:

- episode
- cumulative reward
- episode length
- loss
- epsilon



- average Q
- Q-table size for Flappy Bird

Recommended CSV:

```
episode,  
reward,  
episode_length,  
loss,  
epsilon,  
avg_q,  
score,  
seed
```

For Flappy:

```
episode,  
reward,  
episode_length,  
epsilon,  
score,  
q_table_size,  
seed
```

---

## 47. LOGGING FLOW

```
TRAINING LOOP
|
+--> reward
|
+--> episode length
|
+--> loss
|
+--> epsilon
|
+--> avg Q
|
+--> score
|
v
CSV / JSON
|
v
plots
|
v
report
```

## 48. CHECKPOINTING

Official Milestone 3 requires checkpointing every:

100 episodes

A DQN checkpoint should contain at least:

```
online network
target network
optimizer state
episode
global step
epsilon
configuration
random-state information where practical
```

A tabular checkpoint should contain:

```
Q-table
episode
epsilon
configuration
seed
```

---

## 49. CHECKPOINT FLOW

```
Episode 100
  |
  v
save checkpoint
  |
  v
Episode 200
  |
  v
save checkpoint
  |
  v
Episode 300
  |
  v
save checkpoint
```

The purpose is not only safety.

It enables:

```
TRAIN
  |
  v
STOP
  |
  v
LOAD
  |
  v
RESUME
```

---

## 50. MODEL FILES

Official Milestone 3 deliverables include:

```
models/pacman_dqn_best.pt
models/snake_dqn_best.pt
models/flappy_q_table.pkl
models/flappy_sarsa_table.pkl
```

Also maintain periodic checkpoints:

```
checkpoints/
```

Do not overwrite the only good checkpoint with an experimental run.

---

## 51. GOOGLE COLAB + GOOGLE DRIVE

Colab is useful for the deep-learning training stage.

Conceptual workflow:

```
LOCAL PROJECT
  |
  v
Git repository
  |
  v
Google Colab
  |
  v
Google Drive
  |
  +--> checkpoints
  +--> logs
  +--> models
  +--> plots
```

Use local development for:

- code
- tests
- environment debugging

Use Colab GPU resources for:

- long DQN training
  - repeated experiments
  - model checkpoint generation
- 

## 52. NOTEBOOK STRATEGY

Do not put the whole project into one enormous notebook.

Use notebooks for experiments and visualization.

Put reusable implementation into:

```
src/
```

Recommended notebooks:

```
00_setup_smoke_test.ipynb
```

```
snake/
```

```
01_snake_environment.ipynb
02_snake_preprocessing.ipynb
03_snake_dqn_training.ipynb
04_snake_evaluation.ipynb
```

```
flappy/
```

```
01_flappy_environment.ipynb
02_flappy_state_discretization.ipynb
03_flappy_q_learning.ipynb
04_flappy_sarsa.ipynb
05_flappy_evaluation.ipynb
```

```
pacman/
```

```
01_pacman_environment.ipynb
02_pacman_preprocessing.ipynb
03_pacman_dqn_training.ipynb
04_pacman_evaluation.ipynb
```

---

## 53. PART 6 --- BASELINES

Before claiming that an RL agent learns, establish a non-RL baseline.

At minimum:

```
Random policy
```

For each game:

```
random agent
  |
  v
multiple episodes
  |
  v
baseline score distribution
```

Store:

```
evaluations/<game>/random_baseline.csv
```

---

## 54. FAIR COMPARISON

For algorithm comparisons, hold fixed:

```
environment rules
state representation
reward function
evaluation horizon
scoring criteria
evaluation seeds
```

Only change the algorithm or the factor under study.

Example:

```
Flappy environment
  |
  +--> Q-Learning
  |
  +--> SARSA
```

Both should receive:

same state  
same actions  
same reward  
same environment  
same evaluation protocol

---

## 55. PART 7 --- INITIAL EVALUATION

Official Milestone 3 calls for:

100 evaluation episodes

Compare:

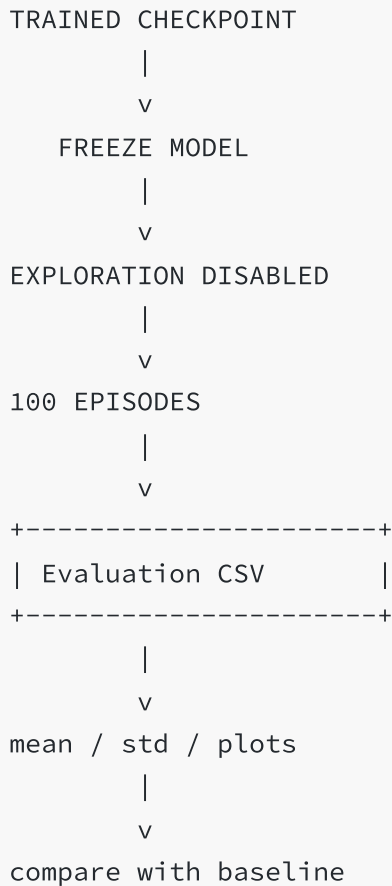
trained agent  
vs  
random agent

For each episode record:

episode  
score  
return  
episode length  
success indicator  
seed

---

## 56. EVALUATION FLOW



## 57. REQUIRED METRICS

Milestone 1 identifies:

### Episode return

Total reward over an episode.

### Native game score

The score provided by the game.

Reward and game score are not necessarily identical.

### Survival / episode length

Important for:

- Snake



- Flappy Bird
- 

## Success rate

Define the success condition before evaluation.

Example:

```
success = survives >= N steps
```

or another game-appropriate criterion.

Do not invent a success criterion after seeing results.

---

## Mean $\pm$ standard deviation

Report:

```
mean score  
std score
```

rather than only:

```
best score
```

---

## Convergence and stability

Inspect:

```
reward curve  
loss curve  
epsilon  
average Q  
score curve
```

---

## 58. PART 8 --- PLOTS

Minimum plots:

1. reward vs episode
2. moving-average reward
3. episode length vs episode
4. score vs episode
5. loss vs training step/episode
6. epsilon vs episode/frame
7. evaluation score distribution
8. algorithm comparison

For DQN:

average Q  
loss

For Flappy:

Q-table size

---

## 59. MOVING AVERAGE

Raw reward can be noisy.

Use a moving average for visualization:

```
raw reward:
/\ \_ /\ \_ /\ \_

moving average:
___/-----\_____
```

Do not replace the raw data. Store both.

---

## 60. PART 9 --- GAMEPLAY RECORDING

Record representative trajectories.

For each game save:

```
random baseline
early training
mid training
final trained agent
failure case
```

Suggested:

```
videos/
├── snake/
│   ├── random.mp4
│   ├── early_training.mp4
│   ├── final_agent.mp4
│   └── failure_case.mp4
├── flappy/
│   ├── random.mp4
│   ├── q_learning.mp4
│   ├── sarsa.mp4
│   └── failure_case.mp4
└── pacman/
    ├── random.mp4
    ├── early_training.mp4
    ├── final_agent.mp4
    └── failure_case.mp4
```

## 61. PART 10 --- FINAL COMPARISON

Do not compare raw scores across unrelated games as if they were the same metric.

Bad:

```
Pac-Man = 500
Snake = 20
Flappy = 10

therefore Pac-Man is better
```

This is meaningless because the games have different scales and objectives.

Instead compare **within each game**:

Snake:

random vs DQN

Flappy:

random vs Q-Learning vs SARSA

Pac-Man:

random vs DQN

Then compare qualitative learning characteristics across games:

state complexity

training stability

sample efficiency

reward sparsity

exploration sensitivity

failure modes

---

## 62. PART 11 --- ABLATION STUDIES

Milestone 3 transitions into controlled comparison and ablation.

Possible ablations:

### Reward ablation

baseline reward

vs

remove survival shaping

vs

modified shaping

Only if the experiment is planned and documented.

---

### State ablation

Example:

4-frame history

vs

single frame

if computationally feasible.

---

## Exploration ablation

Compare documented epsilon schedules.

---

## Replay ablation

For Snake:

dual replay  
vs  
ordinary replay

if the team has enough compute/time.

---

## Algorithm ablation

Flappy naturally provides:

Q-Learning  
vs  
SARSA

---

## 63. PART 12 --- FAILURE ANALYSIS

When an agent fails, record **why** rather than simply saying:

The model did not work.

Classify failures.

#### FAILURE

- |
- +--> environment bug?
- |
- +--> state representation?
- |
- +--> reward problem?
- |
- +--> exploration problem?
- |
- +--> unstable DQN?
- |
- +--> insufficient training?
- |
- +--> game-specific difficulty?

---

## 64. REWARD HACKING

Example:

```
intended:
eat apple

bad reward:
survive forever
```

The agent might learn:

```
avoid risk
but never pursue apple
```

Therefore inspect:

```
native score
reward
trajectory
behavior
```

together.

---

## 65. STATE EXPLOSION

For tabular methods:

```
many possible states
  |
  v
huge Q-table
  |
  v
most entries never visited
  |
  v
slow learning / memory inefficiency
```

This is one reason DQN is appropriate for richer visual states.

---

## 66. OVER-EXPLORATION

If epsilon stays too high:

```
learned policy
  |
  v
random action
  |
  v
poor evaluation
```

Always separate:

```
training epsilon
from
evaluation policy
```

---

## 67. DQN INSTABILITY

Symptoms:

```
loss suddenly explodes
reward collapses
Q-values become extreme
performance oscillates
```

Check:

```
replay buffer
target update
learning rate
reward scaling
gradient handling
state normalization
terminal handling
```

The Milestone 1 risk mitigation specifically emphasizes replay, target networks, conservative learning rate, gradient clipping where required, and repeated seeds.

---

## 68. TERMINAL HANDLING

A common DQN bug is bootstrapping from terminal states.

Conceptually:

```
if done:
    target = reward

else:
    target = reward + gamma * next_Q
```

Do not bootstrap through a terminal state.

For Gymnasium environments, distinguish:

```
terminated
truncated
```

according to the environment semantics and document how the project converts them into its training `done` behavior.

---



## 69. FRAME STACKING

For games where temporal information matters:

```
Frame t-3
Frame t-2
Frame t-1
Frame t
  |
  v
stack
  |
  v
DQN input
```

Why?

A single image may not reveal movement direction.

Four frames can provide temporal information.

---

## 70. RANDOM SEEDS

Use multiple seeds where feasible.

Example:

```
seed 42
seed 123
seed 456
seed 789
seed 2026
```

The exact number of seeds can depend on compute availability, but final claims should not rely on one run.

Store them in:

```
metadata/seeds.json
```

---

## 71. EXPERIMENT REGISTRY

Every experiment should have an identifier.

Example:

```
SNAKE_DQN_SEED42_V1  
FLAPPY_Q_SEED42_V1  
FLAPPY_SARSA_SEED42_V1  
PACMAN_DQN_SEED42_V1
```

Registry:

```
{  
  "experiment_id": "FLAPPY_Q_SEED42_V1",  
  "game": "flappy",  
  "algorithm": "q_learning",  
  "seed": 42,  
  "config": "configs/flappy/q_learning.yaml",  
  "status": "completed"  
}
```

---

## 72. REPRODUCIBILITY CHECKLIST

Record:

```
[ ] Python version  
[ ] package versions  
[ ] environment ID  
[ ] environment version/commit where practical  
[ ] state definition  
[ ] preprocessing  
[ ] action mapping  
[ ] reward constants  
[ ] alpha  
[ ] gamma  
[ ] epsilon schedule  
[ ] episode count  
[ ] frame count  
[ ] seed  
[ ] neural architecture  
[ ] learning rate  
[ ] replay configuration  
[ ] target update interval  
[ ] checkpoint interval  
[ ] evaluation protocol
```

---

## 73. MILESTONE 2 EVIDENCE PACKAGE

Before claiming that the environment is complete, produce:

```
reports/environment_validation.md

logs/
├── snake_environment_smoke_test.csv
├── flappy_environment_smoke_test.csv
└── pacman_environment_smoke_test.csv

observations/
├── snake/
├── flappy/
└── pacman/

reward_traces/
├── snake/
├── flappy/
└── pacman/

episode_traces/
├── snake/
├── flappy/
└── pacman/
```

Required evidence:

```
reset
action
observation
reward
terminal
integration
```

---

## 74. MILESTONE 3 EVIDENCE PACKAGE

Produce:

```
models/  
checkpoints/  
logs/  
plots/  
videos/  
configs/  
metadata/
```

Required artifacts include:

```
models/pacman_dqn_best.pt  
models/snake_dqn_best.pt  
models/flappy_q_table.pkl  
models/flappy_sarsa_table.pkl
```

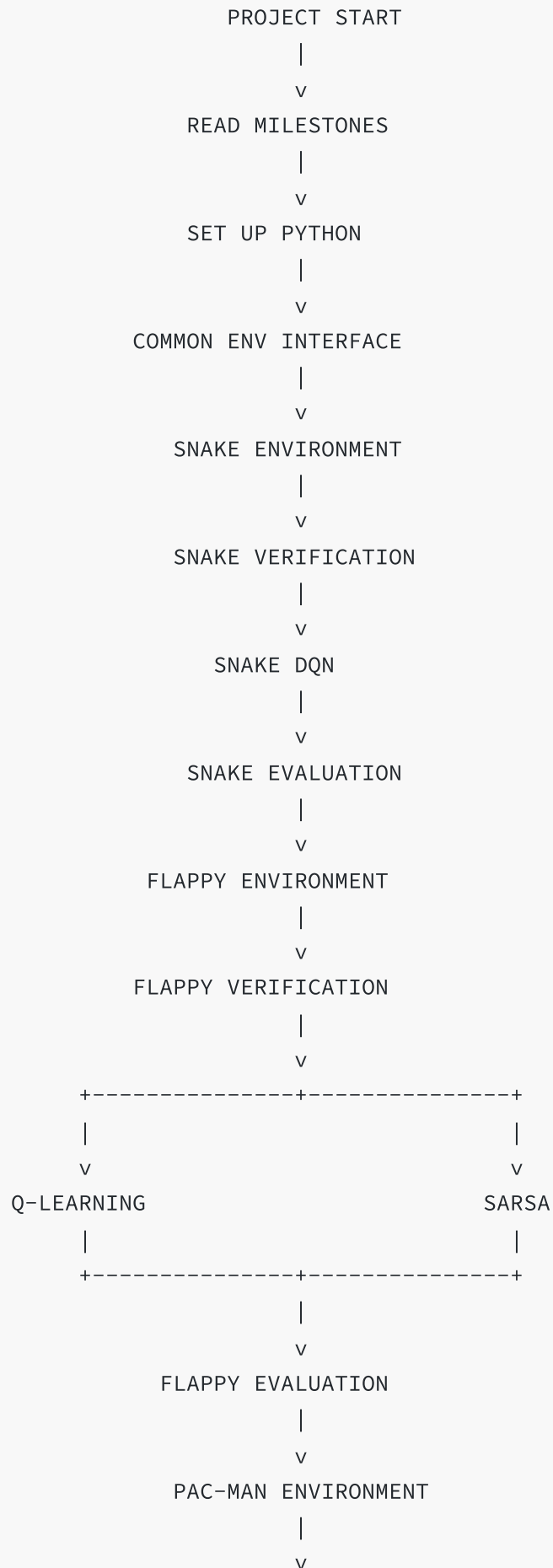
and:

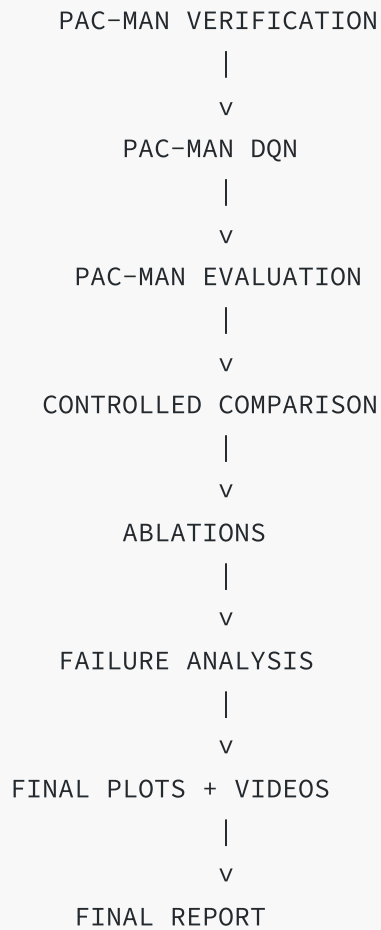
```
training notebooks  
CSV logs  
learning curves  
loss curves  
gameplay videos  
comparison plots  
JSON configs  
metadata  
REPRODUCIBILITY.txt
```

---

## 75. FINAL PROJECT FLOW

The complete project should look like:

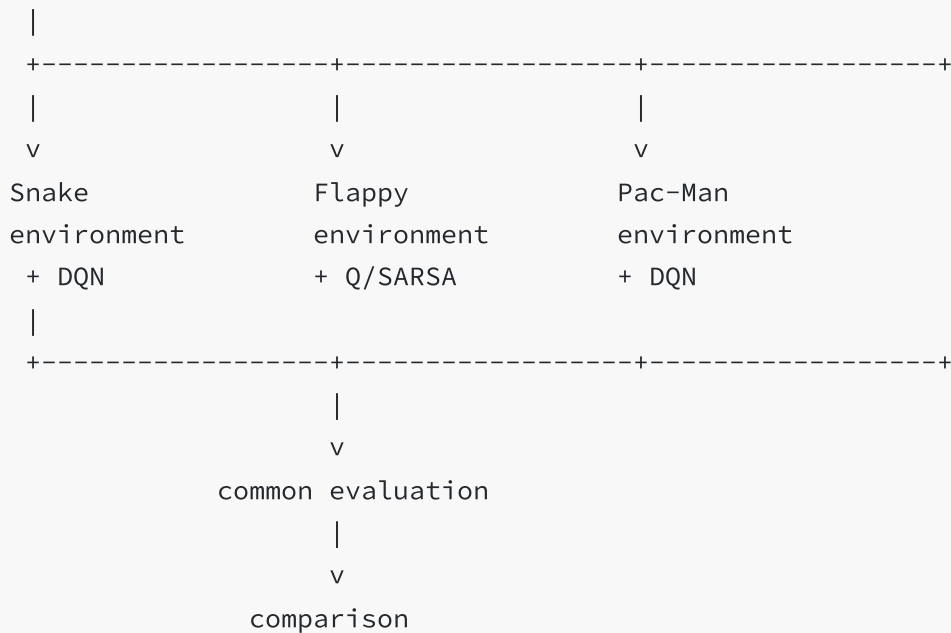




## 76. WHAT EACH TEAM MEMBER CAN WORK ON

A practical parallelization:

TEAM



However, the **common environment interface and logging format should be agreed upon first.**

---

## 77. RECOMMENDED TEAM DEVELOPMENT ORDER

If multiple people are working:

### Phase A

One person:

```
common environment interface
common config format
common logging
common checkpoint format
```

### Phase B

Parallel:

```
Person A -> Snake
Person B -> Flappy
Person C -> Pac-Man
```

## Phase C

Everyone integrates into:

```
common evaluation
common plots
common metadata
```

---

## 78. TESTING STRATEGY

Test at three levels.

### Level 1 --- Unit tests

Examples:

```
reward calculation
state encoding
epsilon schedule
Q update
replay sampling
checkpoint save/load
```

---

### Level 2 --- Environment tests

```
reset
step
terminal
reward
state shape
action legality
```

---



## Level 3 --- Integration tests

```
environment
  |
  v
agent
  |
  v
action
  |
  v
environment
  |
  v
update
  |
  v
logging
  |
  v
checkpoint
```

---

## 79. DEBUGGING ORDER

When training fails, debug in this order:

```
1. Does the game run?
  |
  v
2. Does reset work?
  |
  v
3. Does step work?
  |
  v
4. Is reward correct?
  |
  v
5. Is terminal correct?
  |
  v
6. Is state shape correct?
  |
  v
7. Is action mapping correct?
  |
  v
8. Does random agent work?
  |
  v
9. Does one training update work?
  |
  v
10. Does loss remain finite?
    |
    v
11. Does checkpoint load?
    |
    v
12. Does learning improve?
```

Do not start changing hyperparameters before verifying the lower layers.

---

## 80. RANDOM AGENT DEBUGGING

A random agent is extremely useful.

It should:

```
reset
|
v
random legal action
|
v
step
|
v
record reward
|
v
repeat
```

If the random agent crashes because the environment itself is broken, training cannot be meaningful.

---

## 81. ONE-BATCH DQN TEST

Before full training:

```
collect transitions
  |
  v
fill replay
  |
  v
sample one batch
  |
  v
forward pass
  |
  v
target calculation
  |
  v
loss
  |
  v
backward
  |
  v
optimizer step
```

Verify:

```
loss is finite
gradients exist
weights change
```

Then run a short training test.

---

## 82. CHECKPOINT LOAD TEST

Immediately after saving:

```
save
|
v
create new agent
|
v
load checkpoint
|
v
run evaluation
```

The loaded agent should produce the same behavior as the saved state, subject to controlled randomness.

---

## 83. DATASET TERMINOLOGY

This project does not depend on a conventional static dataset for RL training.

Instead:

```
GAME
|
v
AGENT INTERACTION
|
v
(state, action, reward, next_state, done)
|
v
REPLAY BUFFER / Q-TABLE UPDATE
```

Therefore use terms such as:

```
experience
trajectory
transition
replay buffer
training experience
```

rather than implying that a Kaggle-style dataset is required.

---

## 84. RESOURCE LINKS

### 84.1 Gymnasium

Official documentation:

<https://gymnasium.farama.org/>

Official GitHub:

<https://github.com/Farama-Foundation/Gymnasium>

Use these for:

- environment API
  - reset/step behavior
  - spaces
  - wrappers
  - environment debugging
  - current Gymnasium compatibility
- 

### 84.2 Atari / ALE

Gymnasium Atari documentation:

<https://gymnasium.farama.org/environments/atari/>

ALE Pac-Man documentation:

<https://github.com/Farama-Foundation/Arcade-Learning-Environment/blob/main/docs/environments/pacman.md>

ALE paper:

<https://arxiv.org/abs/1207.4708>

Use these for Pac-Man/Atari environment context.

---

### 84.3 Flappy Bird Gymnasium

Repository:

<https://github.com/markub3327/flappy-bird-gymnasium>

Use it for:

- Flappy Bird environment integration
- action mapping
- environment mechanics
- observation interface

Important:

The repository's default reward values are not automatically the project's reward values.

The project configuration uses:

```
survive      +0.5
pipe         +5
collision    -1000
```

---

## 84.4 DQN Research

Mnih et al.:

<https://arxiv.org/abs/1312.5602>

*Playing Atari with Deep Reinforcement Learning*

Use for:

- DQN background
- CNN-based Q-function approximation
- replay
- Atari visual learning

---

## 84.5 Dueling DQN Research

<https://arxiv.org/abs/1511.06581>

*Dueling Network Architectures for Deep Reinforcement Learning*

Use because the official Milestone 3 Snake and Pac-Man architectures use a dueling head.

---

## 85. RESOURCES REMOVED FROM THE CORE WORKFLOW

The following resources were in the earlier links file but should not be treated as project dependencies.

## Minari

<https://github.com/Farama-Foundation/Minari>

Purpose:

offline RL datasets

Not required because this project trains agents through online interaction.

---

## D4RL

<https://arxiv.org/abs/2004.07219>

Purpose:

offline RL benchmark datasets

Not required for the current project.

---

## CALE

<https://arxiv.org/abs/2410.23810>

Purpose:

continuous-action Atari

Not required because the current project uses discrete game actions.

---

## CartPole

[https://gymnasium.farama.org/environments/classic\\_control/cart\\_pole/](https://gymnasium.farama.org/environments/classic_control/cart_pole/)

Optional only.

Use as a smoke test if the team needs to verify the generic RL infrastructure before the actual games.

Do not include CartPole in the final three-game experimental results unless the project scope is explicitly changed.

---



## 86. RESEARCH BASIS

The supplied project milestones identify these principal research foundations:

### General RL

- Watkins & Dayan --- Q-Learning
- Rummery & Niranjan --- SARSA
- Sutton & Barto --- Reinforcement Learning

### Deep RL

- Mnih et al. --- DQN

### Snake

- Zhepei Wei et al. --- *Autonomous Agents in Snake Game via Deep Reinforcement Learning*

### Pac-Man

- K. Vijaychandra Reddy --- *Playing Pac-Man using Reinforcement Learning*

### Flappy Bird

- Tai Vu & Leon Tran --- *FlapAI Bird: Training an Agent to Play Flappy Bird Using Reinforcement Learning Techniques*

The Milestone 2 environment definitions were intentionally based on the three supplied game-specific papers.

---

## 87. SOURCE FIDELITY RULE

When there is a difference between:

research paper  
and  
official project milestone configuration

do not hide the difference.

Document:

Paper-defined behavior  
+  
Project implementation configuration  
=  
actual experiment

The official Milestone 3 configuration controls the actual implementation when it specifies a project value.

---

## 88. EXPERIMENT MATRIX

The main final experiment should be approximately:

Game Agent Representation Learning type

---

Snake Refined DQN 4-frame visual Deep off-policy Flappy Bird Q-Learning 75 states Tabular off-policy Flappy Bird SARSA 75 states Tabular on-policy Pac-Man DQN 4-frame visual Deep off-policy

Optional experiments should be clearly labeled as additional/ablation experiments.

---

## 89. MAIN HYPOTHESES

Possible project hypotheses grounded in the milestone design:

### H1 --- State representation matters

A richer temporal/visual state can improve behavior in environments where a single compact state is insufficient.

---

### H2 --- On-policy and off-policy learning can behave differently

Flappy Bird provides a controlled comparison:

Q-Learning  
vs  
SARSA

under the same environment and state representation.

---

### H3 --- DQN is useful when tabular representation becomes impractical

Pac-Man and visual Snake provide richer state spaces where explicit state-action tables are not a practical primary representation.

---

### H4 --- Reward shaping affects learned behavior

Compare reward curves and actual gameplay rather than reward alone.

---

## 90. WHAT THE PROJECT SHOULD NOT CLAIM

Do not claim:

"universal game-playing intelligence"

unless a transfer experiment actually demonstrates it.

Do not claim:

"human-level performance"

unless the experiments provide evidence.

Do not claim:

"DQN is on-policy"

It is off-policy.

Do not claim:

"our best episode proves the model works"

Repeated evaluation is required.

---

## 91. FINAL DEMONSTRATION PLAN

A strong final demo should contain:

## Demo 1 --- Snake

```
launch game
  |
  v
load trained DQN
  |
  v
play automatically
  |
  v
show score
  |
  v
show behavior
```

---

## Demo 2 --- Flappy comparison

```
Q-Learning
  |
  v
play

SARSA
  |
  v
play

compare scores / survival
```

---

## Demo 3 --- Pac-Man

```
load trained DQN
  |
  v
processed frames
  |
  v
action selection
  |
  v
gameplay
```

---

## Demo 4 --- Evaluation dashboard

Show:

```
mean score
std score
mean return
survival
success rate
training curve
evaluation distribution
```

---

## 92. FINAL REPORT STRUCTURE

Recommended report:

1. Introduction
  2. Problem Statement
  3. Objectives
  4. Reinforcement Learning Background
  5. MDP Formulation
  6. Common System Architecture
  7. Environment Design
    - 7.1 Snake
    - 7.2 Flappy Bird
    - 7.3 Pac-Man
  8. State Representation
  9. Reward Design
  10. Algorithms
    - 10.1 Q-Learning
    - 10.2 SARSA
    - 10.3 DQN
  11. Implementation
  12. Training Configuration
  13. Experimental Setup
  14. Baselines
  15. Evaluation Metrics
  16. Results
  17. Ablation Studies
  18. Failure Analysis
  19. Reproducibility
  20. Limitations
  21. Conclusion
  22. References
-

## 93. FINAL DELIVERABLE CHECKLIST

### Environment

- ☐ Snake environment
- ☐ Flappy environment
- ☐ Pac-Man environment
- ☐ common interface
- ☐ reset tests
- ☐ step tests
- ☐ reward tests
- ☐ terminal tests
- ☐ state dimension tests

### Agents

- ☐ Snake refined DQN
- ☐ Flappy Q-Learning
- ☐ Flappy SARSA
- ☐ Pac-Man DQN

### Training

- ☐ epsilon-greedy
- ☐ replay
- ☐ target network
- ☐ checkpointing
- ☐ logging
- ☐ seed management

## Evaluation

- ☐ random baseline
- ☐ 100 evaluation episodes
- ☐ score
- ☐ return
- ☐ survival
- ☐ success rate
- ☐ mean
- ☐ standard deviation
- ☐ plots

## Evidence

- ☐ gameplay videos
- ☐ representative trajectories
- ☐ failure examples
- ☐ environment validation
- ☐ configuration files
- ☐ metadata
- ☐ reproducibility report

---

## 94. DEFINITION OF DONE

The project is not done merely because:

the model runs

It is done when:

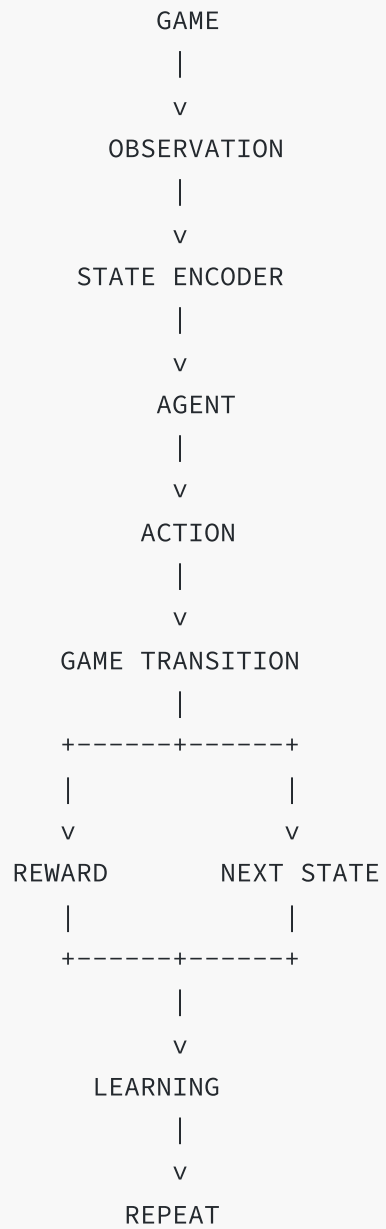


```
Environment works
+
State is correct
+
Reward is correct
+
Agent trains
+
Checkpoint works
+
Logs exist
+
Baseline exists
+
Evaluation is repeated
+
Results are reproducible
+
Failure cases are analyzed
+
Comparison is fair
+
Final evidence is documented
```

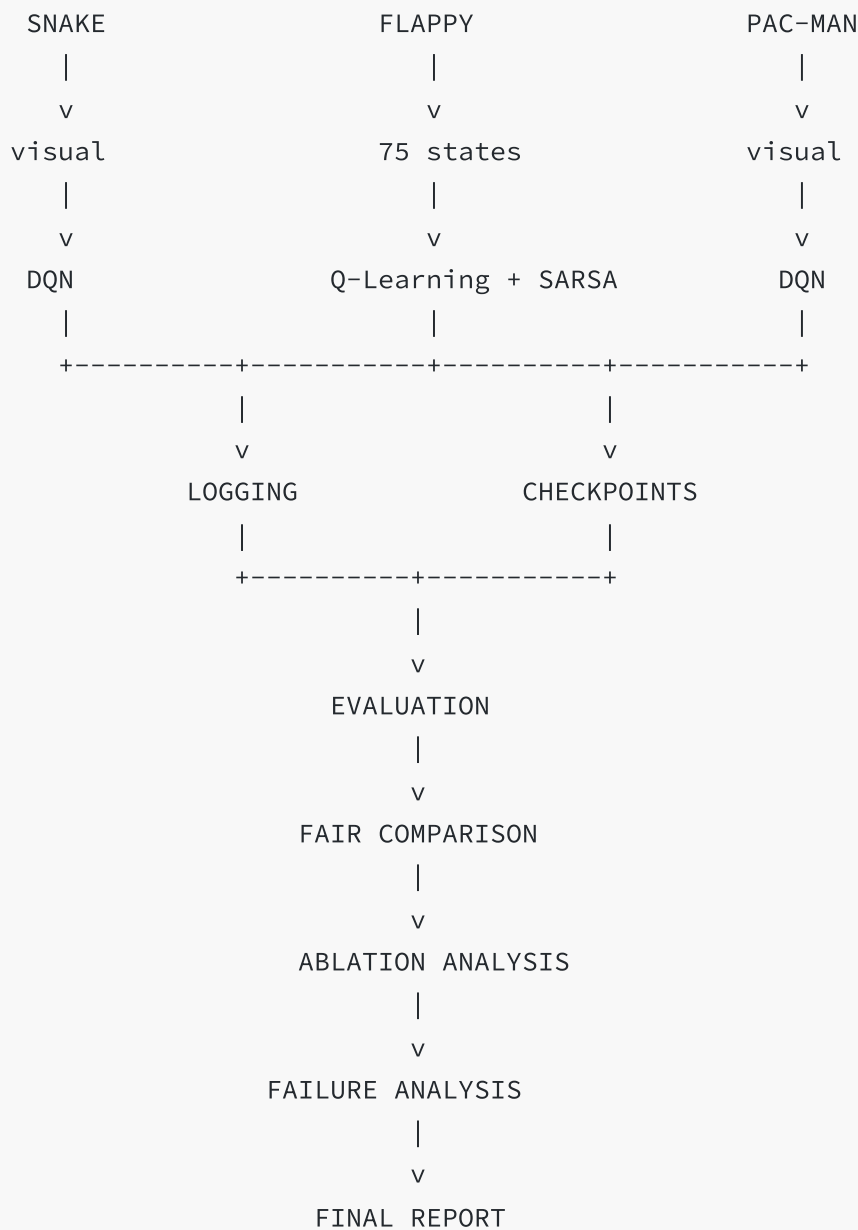
---

## 95. FINAL MENTAL MODEL

Remember this:



For the whole project:



## 96. THE PRACTICAL STARTING CHECKLIST

When you actually begin coding, do **not** start with all three games.

Start here:

```
DAY / SESSION 1
|
+--> create repository
|
+--> create Python environment
|
+--> install core dependencies
|
+--> create folder structure
|
+--> create common config/logging utilities
|
+--> run Gymnasium smoke test
|
v
STOP
```

Then:

```
SESSION 2+
|
+--> build Snake environment
|
+--> verify reset
|
+--> verify actions
|
+--> verify rewards
|
+--> verify terminal
|
+--> verify state preprocessing
|
v
ONLY THEN
|
+--> implement Snake DQN
```

After Snake is stable:

### Flappy Bird

```
|  
+--> environment  
+--> 75-state encoder  
+--> Q-Learning  
+--> SARSA  
+--> comparison
```

After Flappy is stable:

### Pac-Man

```
|  
+--> Atari environment  
+--> preprocessing  
+--> frame stack  
+--> DQN  
+--> evaluation
```

Finally:

```
ALL GAMES  
|  
v  
COMMON EVALUATION  
|  
v  
REPEATED SEEDS  
|  
v  
ABLATION  
|  
v  
FAILURE ANALYSIS  
|  
v  
FINAL REPORT + DEMO
```

---

## 97. FINAL OPERATING RULE

For every component, use this development pattern:

BUILD

|

v

TEST

|

v

LOG

|

v

VISUALIZE

|

v

VERIFY

|

v

ONLY THEN

|

v

TRAIN / SCALE UP

Do not skip directly from:

BUILD

to:

10,000 episodes

The strongest outcome for this project is not a single spectacular game score.

It is a **reproducible experimental system** that clearly demonstrates:

different games  
+  
different state representations  
+  
different RL algorithms  
+  
different reward structures  
+  
controlled training  
+  
repeated evaluation  
+  
failure analysis  
=  
evidence-based understanding of RL for game optimization

---

## 98. SOURCE / RESOURCE INDEX

### Project milestone documents

Game\_Optimization\_Reinforcement\_Learning\_MITS\_Gwalior\_Milestone - 1.pdf  
MITS\_Gwalior\_Game\_Optimization\_RL\_Milestone\_2.pdf  
Milestone\_3\_Final\_Complete.pdf

The official Milestone 3 document supplied by the project team is the controlling source for the final agent configurations and training settings in this guide.

## External resources

Gymnasium:

<https://gymnasium.farama.org/>

Gymnasium GitHub:

<https://github.com/Farama-Foundation/Gymnasium>

Gymnasium Atari:

<https://gymnasium.farama.org/environments/atari/>

ALE Pac-Man:

<https://github.com/Farama-Foundation/Arcade-Learning-Environment/blob/main/docs/environments/pacman.md>

Flappy Bird Gymnasium:

<https://github.com/markub3327/flappy-bird-gymnasium>

DQN:

<https://arxiv.org/abs/1312.5602>

Dueling DQN:

<https://arxiv.org/abs/1511.06581>

ALE:

<https://arxiv.org/abs/1207.4708>

CartPole:

[https://gymnasium.farama.org/environments/classic\\_control/cart\\_pole/](https://gymnasium.farama.org/environments/classic_control/cart_pole/)

---

## 99. END STATE

At the end of the project, the repository should allow a new team member to understand:



WHAT

|

v

we are building

WHY

|

v

each algorithm is used

HOW

|

v

the environment works

HOW

|

v

the agent learns

HOW

|

v

training is configured

HOW

|

v

results are evaluated

WHY

|

v

the conclusions are trustworthy

A new team member should be able to clone the repository, read this guide, inspect the configs, run the environment smoke tests, load a checkpoint, reproduce an evaluation, and understand where each result came from.